Varsel Innlandet: An Agentic Digest of Natural Hazard Warnings for Innlandet

Kim [Surname] **Ådne [Surname]**
KIUA-2003, University of Inland Norway, Hamar
Repository: `https://github.com/zetsukko/weather-demo-live`

Abstract

Emergency coordinators and volunteer groups in Innlandet must follow official flood, landslide and weather warnings that are published by two agencies, in different formats, and revised several times a week. Today the active warnings for the county are collected by hand and retyped into briefings. We propose a small agentic harness that fetches the warnings from the NVE and MET Norway open APIs, lets a language model write a short plain-Norwegian summary of each warning, and assembles a daily digest whose factual content is checked by code before anyone reads it. The central design choice is that the model writes prose only: warning identifiers, danger levels, areas and dates are copied deterministically from the source, and a checker rejects any digest that drops, duplicates, alters or invents a warning. This report describes the problem, a binary definition of success, the architecture, a schedule to the demonstration, and the main risks.

1 Problem

Recurring need. Innlandet is Norway's largest county by area, with river systems such as Glomma, Gudbrandsdalslågen and Mjøsa, and a mountain interior. Flood and landslide warnings from the Norwegian Water Resources and Energy Directorate (NVE) and weather warnings from MET Norway are therefore issued for the county repeatedly every autumn and spring. Municipal emergency coordinators (*beredskapskoordinatorer*) and volunteer organisations such as the Red Cross, Norwegian People's Aid and local search-and-rescue teams need an up-to-date picture of which warnings apply to their area, at which level, and for which period. Today that picture is assembled manually by checking varsom.no and MET's warning pages and copying the relevant warnings into an email or briefing note. The work is repetitive, it has no single owner, and its failure modes are costly: a missed update, a cancelled warning still being circulated, or a wrong danger level.

What the workflow produces. Once a day, the harness produces `out/digest.json` and a readable `out/digest.md`. Each entry contains the official warning identifier, type, danger level, affected area and validity period, copied from the source, plus a summary of at most 60 words in plain Norwegian written by the agent. The digest carries the attribution required by the data licences and links to varsom.no. A person reads the digest and decides whether and how to pass it on; the system never sends messages.

Users and constraints. Users are described by role only. The workflow processes public, official warnings and stores no personal data. Both sources are open: NVE data is licensed under NLOD 2.0

and MET Norway data under NLOD 2.0 and CC BY 4.0, both requiring attribution. We follow MET's terms (identifying `User-Agent`, caching) and NVE's request not to selectively remove parts of a warning: the official fields are always kept intact, and the agent's text is labelled as our summary.

2 Definition of success

A run is successful if and only if `python eval/check_digest.py` exits with code 0. The checker compares the digest with the raw source response saved in `fixtures/` for that run and enforces the following binary properties:

1. **Completeness:** every active warning for Innlandet in the source (danger level ≥ 2, valid in the period) appears in the digest exactly once, identified by its source ID.
2. **No invention:** every `warning_id` in the digest exists in the source.
3. **Fidelity:** for each warning, `danger_level`, `valid_from` and `valid_to` are equal to the source values.
4. **Locality:** every warning in the digest lists Innlandet (county number 34) among its counties.
5. **Grounded summaries:** every date, time and number that appears in a `summary_no` also appears in the source record for that warning, and each summary is at most 60 words.
6. **Attribution and privacy:** the digest contains the NVE and MET attribution strings and no e-mail addresses or phone numbers (regular-expression check).
7. **Empty case:** if the source contains no active warning, the digest states this explicitly and contains no warnings.

Properties 1–7 are mechanically decidable. Whether a summary is *useful* and *readable* is not; for the final evaluation we will add a documented LLM-as-judge rubric for these qualities, reported separately from the binary checks.

3 Architecture

The harness is a pipeline of five stages (Figure 1). Only Stage 3 uses a language model; every other stage is deterministic Python using the standard library.

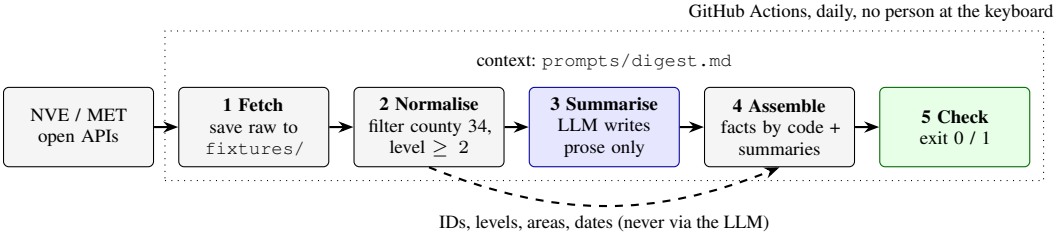


Figure 1: Pipeline of the harness. The dashed path carries all facts around the language model.

Stage 1: Fetch (tool use). `harness/fetch.py` calls the NVE flood-warning API (`https://api01.nve.no/hydrology/forecast/flood/v1.0.10`), endpoint `/Warning/County/34/1/{start}/{end}`, with an identifying `User-Agent`. Landslide warnings and MET MetAlerts (`metalerts/2.0/current` with `county=34`, `geographicDomain=land`) are added after the skeleton. The raw response is written to `fixtures/`, and `--offline` replays a saved fixture, so every run is reproducible without network access.

Stage 2: Normalise. The raw records are mapped to one schema (`source`, `warning_id`, `type`, `danger_level`, `area`, `valid_from`, `valid_to`, official text fields) and filtered to warnings

that list county 34 and have level ≥ 2. This list is the single source of truth for all facts in the digest.

Stage 3: Summarise (agent step). One call to Claude Haiku through the Anthropic API receives the normalised list and a context file, `prompts/digest.md`, which defines the audience, the plain-language rules, the meaning of the NVE activity levels and the required JSON output (`warning_id` $\rightarrow$ `summary_no`). The model sees only the normalised warnings for Innlandet, not the raw API dump or other counties, which keeps the context small and limits the surface for instructions embedded in warning text. Official text is passed inside clearly delimited data blocks, and the prompt states that it is data, not instructions.

Without an API key, or with `--no-llm`, the stage falls back to deterministic template summaries built from the official fields, and `--offline` replays a saved model answer from `fixtures/`, so the full pipeline is reproducible without network or key.

Stage 4: Assemble. `harness/assemble.py` joins the model's summaries to the Stage 2 records by `warning_id` and writes `out/digest.json` and `out/digest.md` with attribution. The model's output is never trusted for facts; a summary for an unknown ID is discarded and a missing summary fails the check.

Stage 5: Check. `eval/check_digest.py` implements the properties of Section 2. The check was committed before the code it tests and is protected from the coding agent by a `deny` rule on `Edit(/eval/**)` in `.claude/settings.json`.

Automation. `python harness/run.py` runs Stages 1–5 with one command. A GitHub Actions workflow runs it on a daily cron schedule, stores the digest as a build artifact, and fails visibly when the check fails. The API key is held as a GitHub secret and in a git-ignored `.env` locally. A kill switch stops the run without a digest if the API fails, the model returns invalid JSON, or more than 60 active warnings appear; `run.py` deletes stale output first so an old digest cannot fake a pass. Publication remains a human decision: the digest is never sent anywhere automatically.

Development context. `CLAUDE.md` tells the coding agent how to run the checks and states the rules (no edits to `eval/`, no personal data). Each unit of work is specified in `tasks/NNN-*.md` with an acceptance check, implemented on a branch, and merged through a reviewed pull request; corrections of agent output are recorded in `docs/prompt-log.md`. Task 001 (a probe of MET Norway's forecast API) has been completed this way and established the fixture and `--offline` pattern reused here.

4 Schedule

The schedule works backwards from the demonstration in weeks 46–47 and the white paper deadline on 3 December.

When	Milestone
30.10 (C2)	Walking skeleton: NVE flood warnings $\rightarrow$ normalise $\rightarrow$ Haiku summary $\rightarrow$ digest $\rightarrow$ check, via `run.py` and a daily GitHub Actions run. Report submitted.
Week 45	Add NVE landslide warnings and MET MetAlerts. Build test set v1 from saved fixtures: typical days, the empty case, a level-4 warning, multi-county warnings, and a non-Innlandet warning that must be rejected.
Week 46	Safe operation: per-run token cap, kill switch, permission scoping for the runtime. First stress tests (input perturbation, dependency failure). Demo rehearsal.
Weeks 46–47	Live demonstration.
Week 48	Repairs with regression evidence through the evaluation harness; address demo feedback; LLM-as-judge rubric for readability.
3.12	White paper submitted; commit tagged `exam`.

5 Risk assessment

Risk	Mitigation
No active warnings in Innlandet during testing	Test from saved fixtures, including archived date ranges with warnings; the empty case is a required property (Section 2, item 7).
LLM changes a level or date, or invents a warning	Facts never pass through the model (Figure 1); property 5 rejects ungrounded numbers in summaries; a failed check means no digest.
Prompt injection in warning text	Warning text is passed as delimited data; output must match a fixed JSON schema; tested as an input perturbation in week 46.
API change, outage or rate limiting	Version-pinned endpoint (NVE keeps old versions for at least six months), fixtures and `--offline`, explicit error on non-200 responses instead of a partial digest.
Cost or key leakage	Key only in `.env` (git-ignored, read-denied for the coding agent) and GitHub secrets; one model call per run with a fixed `max_tokens`.
Time: two people, five weeks	Thin end-to-end skeleton first; sources are added one task at a time; MetAlerts is dropped before the evaluation work if time runs short.

6 Repository

All code, context files, task specifications and checks are in `https://github.com/zetsukko/weather-demo-live`. The `README.md` explains how to reproduce a normal run (`python harness/run.py`) and an offline run from fixtures (`python harness/run.py --offline`).

A Prompt log (excerpts)

The full log is in `docs/prompt-log.md`. The following entries show agent output that we corrected or rejected.

- **2026-10-01, task 001 plan: missing contact in the User-Agent.** The agent's plan for the MET Norway probe used a default `User-Agent` without contact information, which MET's terms of service require. We rejected the plan and had the agent use the repository URL instead: `weather-demo-live/0.1 github.com/zetsukko/weather-demo-live`. The code passed the check either way; only reading the plan caught the problem.
- **2026-10-01, unrequested push.** While scaffolding the repository, the agent pushed to GitHub without being asked. We now review `/diff` before every commit and push ourselves.
- **2026-10-01, credential in the git remote.** The agent noticed a GitHub token embedded in the `origin` remote URL and printed it in the session. We revoked the token and switched to credential-manager login. This confirmed the need to keep secrets out of anything the agent can read or print.
- **2026-10-26, specification contradicted the assignment.** The agent-drafted `SPEC.md` listed scheduling as out of scope, although the technical minimum requires an automatic part. We moved the daily run into scope and kept only hosting and notifications out.
- **2026-10-26, wrong licence.** The same draft gave MET Norway's licence as CC BY 4.0 only; MET's licence page states NLOD 2.0 and CC BY 4.0. Corrected after checking the source.
- **2026-10-26, facts kept away from the model.** Rather than accept a design where the model writes the whole digest, we required that identifiers, levels, areas and dates are copied by code and that the check rejects any number in a summary that is not in the source.

B Meeting reports

Meeting 2026-10-15. *Attendees:* Ådne, Kim. *Done:* repository scaffolded and tagged `w40-scaffold`; task 001 (MET Norway probe for Hamar) merged via pull request #1 with the check passing; prompt log started; three problem candidates drafted. *Decisions:* shortlist of a hazard warnings digest (NVE), council agenda summaries (Hamar kommune) and a campus bus brief (Entur). *Next:* verify source links, book a discussion hour, choose a problem.

Meeting 2026-10-26. *Attendees:* Ådne, Kim. *Done:* remaining source links and licences verified; `SPEC.md` drafted for candidate 1; problem approved by the lecturer. *Decisions:* hazard warnings digest chosen. The council agenda candidate was dropped because the documents may contain personal data and no reuse licence was found; the bus brief was dropped because real-time data is hard to check against a fixed source. The skeleton starts with NVE flood warnings only; Claude Haiku for the summary step; GitHub Actions for the daily run. *Next:* tasks 002–004 (NVE probe, summary step, `run.py` and schedule) before 30.10.